Agility & Leadership Improvement Plan: Once-Only Technical System (OOTS)

Author: Nelson Ferreira

Date: November 2026

Contents

1 Diagnostic **3**
1.1 Business Context . . . 3
1.2 Analytical Frame: the 7-S Model . . . 3
1.3 Hard Dimensions . . . 4
1.3.1 Strategy . . . 4
1.3.2 Structure . . . 4
1.3.3 Systems . . . 5
1.3.4 Talent Practices . . . 6
1.4 Soft Dimensions . . . 7
1.4.1 Culture . . . 7
1.4.2 Trust . . . 7
1.4.3 Psychological Safety . . . 8
1.4.4 Leadership Behaviours . . . 8
1.5 Summary of the Diagnosis . . . 9

2 Improvement Plan **9**
2.1 Intervention 1: Shared and Coaching Code Review . . . 10
2.2 Intervention 2: Knowledge Transfer Before Contract End . . . 10
2.3 Intervention 3: Psychological Safety Pulse Survey . . . 11
2.4 How the Interventions Work Together . . . 12

3 Leadership Reflection **12**

References **14**

1 Diagnostic

1.1 Business Context

I work as an independent contractor for DIGIT-EU, the Directorate-General for Digital Services at the European Commission. My work is on the Once-Only Technical System (OOTS), a pan-European infrastructure that lets public authorities exchange documents securely across borders. Citizens and businesses then do not have to submit the same information repeatedly [1].

OOTS retrieves data directly from the official source, with the user's explicit consent. It rests on four pillars: a **decentralized architecture** connecting existing national systems; **interoperability enablers** such as eID and eDelivery [2]; **user control and consent**; and **standardized evidence** [3].

OOTS is not a commercial enterprise and does not compete in a market. Its mission is to promote cooperation among sovereign nations and serve the public interest.

Environment (VUCA/BANI). The environment is volatile and complex [11]. Political and legal factors dominate: twenty-seven Member States with different governments and timelines, plus the GDPR, eIDAS, the Once-Only Regulation and national laws. Each exchange must satisfy both Member States involved. The factors are interdependent. A political decision to strengthen data protection creates legal requirements, which demand technological solutions that citizens must accept. This is the non-linear dynamic that BANI describes [12]. Cybersecurity threats add brittleness and anxiety to a system handling sensitive data.

The agility need. Agility here does not mean market speed. It means:

- adapting to regulatory change;
- absorbing the feedback of many sovereign stakeholders;
- releasing reliably in iterations (RFCs and TDDs [4]);
- learning quickly when assumptions prove wrong.

The need is high, because the direction is largely implicit and decisions are often reactive. The environment will not become simpler. Reusability for initiatives such as the EU Digital Wallet will widen the scope further.

1.2 Analytical Frame: the 7-S Model

I group the dimensions with the McKinsey 7-S model [18]. Its hard elements are Strategy, Structure and Systems. Its soft elements are Shared values, Style, Staff and Skills. Strategy, Structure and Systems are analysed below as hard dimensions. Talent practices cover Staff and Skills. Culture covers Shared values, and leadership behaviours cover Style.

1.3 Hard Dimensions

1.3.1 Strategy

OOTS has no formal strategic statement, but its documentation reveals a clear direction [5].

- **Mission:** seamless, secure, consent-driven cross-border data exchange, so nobody provides the same information twice.
- **Vision:** a Union where digital interoperability is the norm and public services work as one system regardless of borders.
- **Values:** decentralization, trust and security, interoperability, reusability and cooperation.

My assessment. The purpose is strong and gives a stable backbone. The weaknesses are:

- there are no measurable outcomes, for example procedures supported or Member States connected by a date;
- there is no periodic strategic review. Stakeholder analysis is not formally used, so the strategy is implicit rather than guided;
- some stakeholders, notably data protection authorities and citizens, are not systematically engaged, although they strongly influence legitimacy and trust.

Strategy is collaborative and iterative, which supports adaptability. It lacks the explicit goals and short feedback cadence that would make it agile.

1.3.2 Structure

My project team is organized as follows (Table 1).

The team has about 20 people in six functional groups. Each group has a lead, and a single Tech Lead sits at the top.

My role is Application Architect and Backend Team Lead. The business rules of the project are dense and sensitive, so the backend is its most complex and technical part. I have four core responsibilities:

- advising the Solutions Architect on technical decisions;
- leading the backend team and securing delivery within milestones;
- implementing the core features and libraries;
- reviewing all backend code assuring its quality;

My assessment.

- **Strengths:** small, specialised, cross-functional teams, with security, compliance and QA embedded in the project rather than external gates. Roles are clear. The team is not buried in a bureaucratic hierarchy, in contrast with the wider Commission.

Team	Composition
Common Service Leader	1 Solutions Architect / Tech Lead
Business Analyst Team	1 BA Lead, 1 Senior BA Analyst
Infrastructure:	
DevSecOps Team	1 DevSecOps Lead, 2 DevSecOps Experts
	1 Vulnerability/Incident Analyst, 1 Risk/Compliance Expert
Development:	
Backend Team	1 Application Architect / BE Team Lead (me)
	2 Senior BE Developers
Frontend Team	1 Tech Business Analyst / FE Team Lead
	2 Senior FE Developers
Quality Assurance:	
Quality Assurance Team	1 QA Team Lead
	2 Senior QA Developers

Table 1: Structure of the project team

- **Weaknesses:** there is a hierarchy of leads. Decisions flow through the Tech Lead, with my advisory input. Reviewing all backend code makes me a potential bottleneck and a single point of failure. Specialisation by function (BA, BE, FE, QA) risks hand-offs instead of fully cross-functional ownership. Larger structures around the project, such as Member States and the Commission, are outside the team's control.

1.3.3 Systems

My assessment. The toolchain is modern and supports agile practice: version control, automated pipelines and cloud deployment enable frequent, reliable releases. Argo CD gives continuous visibility of what is deployed against what is declared in Git, and Prometheus and Grafana make the runtime behaviour of the system observable through metrics, dashboards and alerts. Together they shorten the feedback loop between a release and its effect in production. The code is open source, which adds transparency [6]. Weaknesses to probe:

- email remains a communication channel in a distributed team;
- documentation in Confluence can drift from the code;

Purpose	Tool
Communication	Microsoft Teams and Outlook
Documentation	Confluence
Project management	Jira
Version control	GitLab
Automation	CI/CD pipelines
Deployment	AWS / Azure
Deployment monitoring	Argo CD (GitOps continuous delivery)
Observability	Prometheus (metrics) and Grafana (dashboards and alerting)

Table 2: Project systems

- the dashboards cover system health, but it is not clear that delivery metrics, such as DORA metrics, are tracked to show how fast the organization learns [14].

1.3.4 Talent Practices

Recruitment. DG DIGIT uses a hybrid model with three contractual frameworks [7], [8]:

- **Permanent staff** are recruited through EPSO. Candidates must pass strict reasoning, situational and technical competency assessments to reach AD or AST grades.
- **Temporary staff** are recruited through CAST. IT specialists register in a permanent pool, and are shortlisted for interviews when OOTS needs specific expertise. Contracts run from 1 to 6 years.
- **External consultants** come through IT framework contracts (outsourcing). Private consulting firms win EU tenders and allocate their own engineers and experts to the project. Contracts run from 1 to 4 years. Much of the development, testing and maintenance is delivered this way, and I am part of this group.

Assessment. The hybrid model gives access to scarce skills and flexible capacity. It also brings risks:

- fixed-term contracts mean knowledge walks out the door;
- the team mixes people with different employers, incentives and loyalties;
- selection through EPSO and tenders measures formal competence but not agility traits such as learning and collaboration.

Retention. From my personal experience, four factors keep people:

1. **Compensation.** Salaries are very high. In my view this is the main reason people stay.

2. **Project culture.** Unlike the bureaucratic nature of the EC, the team uses modern development approaches. People focus on designing solutions rather than on dead-end meetings, and they are encouraged to suggest better practices and new ways of working.
3. **Sense of purpose.** People feel they contribute to something bigger than themselves.
4. **Supportive team and recognition.** People help each other and share knowledge. Good consultants are appreciated and can stay longer.

Assessment. Retention is strong and partly intrinsic (purpose, autonomy, community), not only financial. Two caveats:

- the pay effect could be fragile if the market changes;
- retention is tied to recognition of individual consultants, and it is not clear that development and career paths are structured.

1.4 Soft Dimensions

1.4.1 Culture

Culture here has two layers: the Commission's bureaucratic environment and the project's own modern, engineering-driven culture. The project culture values innovation, focus on the work, knowledge-sharing and cooperation. These are consistent with the values of the strategic statement: cooperation, trust and security, and reusability. Decentralization and respect for sovereignty also shape culture, with consensus-building (RFCs) preferred to imposed decisions. The culture is collaborative and open (open source). Its risk is that consensus can slow decisions, and the surrounding institution pulls back towards bureaucracy.

1.4.2 Trust

Trust is a precondition for delegation, fast decisions and experimentation. It differs from psychological safety: trust is about relying on others' competence, benevolence and integrity [13], while psychological safety is about being safe to speak up [9]. Five aspects matter here.

1. **Trust in the product.** Trust and security are core values: every exchange is encrypted, authenticated and auditable. Member States connect only if they trust OOTS with sovereign data, and open source and the RFC process build that trust through transparency.
2. **Peer trust.** People help each other all the time. When someone struggles, any member of any team offers help and pairs or meets to solve the problem. For example, QA members meet BE or FE members to solve bugs proactively. This points to high trust across functional teams.
3. **Trust in me as Backend Lead.** I believe the team trusts me, perhaps because of my work ethic and persistence in finding solutions, but I have not asked them. The

fact that I review all backend code may also signal that I do not delegate enough.

4. **Contractors and EC staff.** We have no choice but to trust each other, and interests are aligned: the project's success matters to both, and EC staff who delegate poorly risk the outcome.
5. **Misunderstandings.** In four years I noticed only one or two misunderstandings, both minor setbacks.

Trust is high, though based on my observation rather than data. Part II can test it, for example by sharing code review with the two seniors.

1.4.3 Psychological Safety

Edmondson [9] defines psychological safety as a shared belief that the team is safe for interpersonal risk-taking. It combines with high standards to create a learning zone. I rate it medium to high, but I have not measured it.

1. **Evidence of safety.** People are encouraged to suggest better practices, share knowledge freely and ask for help. Leaders are open to feedback, and meetings focus on solving problems rather than defending positions.
2. **Contractor and employer risk.** Contractors depend on client recognition to stay, which may discourage them from challenging EC decisions. A team made up of several employers may also hide problems to protect its own firm.
3. **Early reporting.** Security work depends on early reporting of weaknesses. People must feel safe to raise problems early, but this is not verified.
4. **My code reviews.** I am the only reviewer of backend code, and the team has two senior developers, not juniors. They sometimes challenge my reviews, but not often enough. Even experienced developers may defer to my role, so my tone matters.
5. **Verdict.** Safety is medium to high, with one gap: challenge of my reviews is too rare. Part II proposes a short pulse survey (Edmondson's seven items) and an explicit invitation to challenge my reviews.

1.4.4 Leadership Behaviours

Leaders on the project are versatile and open to feedback. Five behaviours stand out.

1. **Openness to feedback.** When the Solutions Architect must make a major technical decision that affects my work, he almost always asks for my opinion, gives me the context and asks me to propose solutions. We then go back and forth until we agree on a final solution. This is leadership by setting context and sense-making, not by command and control.
2. **Decision-making.** The Tech Lead keeps the final decision, but it is reached through dialogue with the people affected. This helps people understand the reasons for decisions, although the back and forth can be slower than a direct decision.
3. **Support and recognition.** Leaders let the team focus on design instead of unproductive meetings, invite suggestions for new practices and appreciate good work. It is not clear that they also structure development and career paths.

4. **My own behaviour.** I combine advisory, coaching and hands-on roles. However, I review all backend code and do not delegate enough, and the two seniors may defer to my role. Whether my reviews coach or only gatekeep matters for the change I propose.
5. **Overall.** Leadership is medium to high. Openness is the clear strength, while my limited delegation is the main gap, which Part II tackles by sharing code review with the two seniors.

1.5 Summary of the Diagnosis

Dimension	**Readiness**	**Main evidence**
Strategy	Medium	Strong purpose, but no measurable objectives or periodic review
Structure	Medium-High	Small, clear, cross-functional teams, but hierarchy of leads and review bottleneck
Systems	High	Modern DevOps toolchain, open source
Talent practices	Medium	Hybrid model gives flexibility, but turnover risk; strong retention
Culture	High	Modern, collaborative, innovation-oriented
Trust	High (unmeasured)	Peer help across teams, aligned interests, few misunderstandings
Psychological safety	Medium-High (unmeasured)	Encouragement to speak up, but contractor and review risks unverified
Leadership behaviours	Medium-High	Open, collaborative decisions, but limited delegation by me

Table 3: Summary of the agility readiness diagnosis

Overall. The project is moderately ready for agility. Its main gaps are the implicit strategy, the code-review bottleneck, knowledge loss at contract end and the unmeasured depth of psychological safety. These gaps shape the interventions proposed in Part II.

---

2 Improvement Plan

The diagnostic shows a team with a strong purpose, a modern toolchain and good peer trust. Its gaps are a review bottleneck centred on me, knowledge that leaves with contractors, and psychological safety and trust that nobody has measured. I propose three interventions. Two are within my own scope as Backend Lead and Application Architect,

and the third measures the effect of all of them. Owners and timelines are my proposals and need to be agreed with the Tech Lead.

2.1 Intervention 1: Shared and Coaching Code Review

Type: leadership practice.

Rationale (Problem-Solution fit). I review all backend code. That creates a bottleneck (Structure), hints that I hold on to control (Trust, Leadership), and gives the two senior developers little reason to push back (Psychological Safety). Sharing the load speeds up delivery and signals confidence in the seniors. Reviews that explain their reasoning teach as well as check, and make disagreement easier [15].

Actions.

- The two seniors become full reviewers of backend code. I review only critical changes, such as security, core business rules, and architecture changes.
- Each review comment explains the reason, not only the verdict. Authors are invited to disagree.
- I ask the seniors to review my own code, so that challenge goes both ways.

Measures of success.

- Share of backend pull requests reviewed by someone other than me: from about 0% to at least 70% within 3 months.
- Number of substantive challenges to my reviews or code per month, expected to rise.
- Defect rate after merge stays stable, to confirm that quality does not drop.

Ownership and schedule. This one is mine. Month 1: agree the review rules, the list of critical areas and a checklist with the seniors. Months 2–3: run the new model and give feedback on review comments. Month 4: retrospective and adjustment.

Launch and persistence.

- *Start:* announce the change myself at a team meeting, explain why and the urgency behind it [17], and begin with one module as a pilot.
- *Formal:* write it into the review checklist and the definition of done.
- *Informal:* step back visibly, thank people who challenge me, and let the seniors act as champions.

2.2 Intervention 2: Knowledge Transfer Before Contract End

Type: talent management adjustment.

Problem addressed. Most of the team are contractors, and contracts end. People stay for recognition, and career paths are unclear, so departures are likely. A few people, myself included, hold critical knowledge, and it leaves with them. Given the dense and sensitive business rules, continuity is at stake.

Actions.

- A short handover checklist for each backend module: architecture notes, key decisions recorded as ADRs, runbooks and open issues.
- Up-to-date module documentation becomes part of the definition of done.
- Pairing between a departing contractor and a successor during a handover period, and pairing on reviews between the seniors and other developers.

Measures of success.

- Bus factor, or truck factor [16], of at least 2 for every backend module.
- Share of modules with current documentation and a handover checklist: at least 90% after 6 months.
- Onboarding time for a new backend developer until the first merged pull request.

Responsibility and schedule. As Backend Team Lead I am accountable, and the Tech Lead steps in when a handover depends on contract changes. Month 1: checklist and documentation template, and a bus-factor map of all modules. Months 2–3: documentation of the most critical modules and a first pairing round. Then a bus-factor review every quarter.

Launch and persistence.

- *Start:* begin with the most critical modules, and use the next contractor departure as a pilot.
- *Formal:* add the checklist to the definition of done and to the offboarding steps.
- *Informal:* I document my own modules first and share my knowledge openly, so that it becomes normal. This also supports Intervention 1.

2.3 Intervention 3: Psychological Safety Pulse Survey

Type: organizational and management tool.

Gap addressed. My ratings of trust and psychological safety rest on observation alone, and without data I cannot judge whether the other two interventions work. A short survey, repeated over time, replaces impressions with evidence and serves as the yardstick for the whole plan.

Actions.

- A short anonymous survey based on Edmondson's seven-item psychological safety scale [10], plus two or three questions on trust and on how reviews are received.
- Run three times: a baseline now, after 6 months, and after 1 year.
- Results are shared with the whole team and discussed openly, with agreed actions.
- Results are reported by group, only if groups are large enough to stay anonymous.

Measures of success.

- Response rate of at least 80%.

- Increase in the average score between baseline and the 1-year round, especially on the items about disagreeing and raising problems.
- Score on the item about disagreeing with others, used as an indicator for Intervention 1.
- At least one action agreed and completed after each round.

Sponsorship and calendar. I sponsor and run the survey, with the Tech Lead's agreement because it reaches staff of several employers. Month 1: design, anonymity rules and baseline. Month 7 and month 13: follow-up rounds. The team discusses each round within two weeks.

Launch and persistence.

- *Start:* explain the purpose to the team, stress that the survey is anonymous, and publish my own commitments from the baseline results.
- *Formal:* keep the survey as a standing item every six months, alongside the retrospective.
- *Informal:* react constructively to low scores. How I respond to the first results will decide whether people answer honestly the next time.

2.4 How the Interventions Work Together

The pulse survey does not depend on the other two interventions, so the baseline can start immediately and gives a "before" picture. It also measures their effect: a rise in scores on disagreeing and raising problems would indicate that shared and coaching review is working. Its results may in turn reveal further gaps to address. Shared review also spreads knowledge across the seniors, which supports Intervention 2. The survey follows its own schedule (baseline, 6 months, 1 year), so the quarterly checkpoints of Interventions 1 and 2 do not coincide with its rounds.

---

3 Leadership Reflection

The diagnostic shows that I am part of the bottleneck I describe. I review all backend code and do not delegate enough, so the change starts with my own habits. I own Interventions 1 and 2 and sponsor the survey, but I cannot change contracts or strategy. There I can only influence, which matters more than authority across several employers.

Three behaviours need to grow. First, **delegating**: handing review to the two seniors and accepting that they will do it differently. Second, **coaching**: explaining the reason behind each review rather than only approving or rejecting. Third, **inviting challenge**: asking for disagreement and welcoming it, since this is what makes psychological safety real.

My plan is to step back from routine reviews in the first three months, ask the seniors to review my code, and hold short monthly one-to-ones. I will judge progress with the pulse survey, the share of pull requests reviewed by others, and informal feedback from

the seniors and the Solutions Architect, with a self-review at 6 months and 1 year. If the results are low, I will discuss them openly and adjust.

Agility here depends less on new tools than on a lead who lets go.

References

[1] European Commission. *About OOTS*. Retrieved from https://ec.europa.eu/digital-building-blocks/sites/spaces/OOTS/pages/610468075/About+OOTS

[2] European Commission. *Architecture*. Retrieved from https://ec.europa.eu/digital-building-blocks/sites/spaces/OOTS/pages/604504755/Architecture

[3] European Commission. *OOTS — Once-Only Technical System*. Retrieved from https://interoperable-europe.ec.europa.eu/collection/digital-building-blocks/solution/once-only-technical-system-oots

[4] European Commission. *OOTS Technical Design Documents Previous Releases*. Retrieved from https://ec.europa.eu/digital-building-blocks/sites/spaces/TDD/pages/887386483/OOTS+Technical+Design+Documents+Previous+Releases

[5] European Commission. *Two systems, one vision*. Retrieved from https://ec.europa.eu/digital-building-blocks/sites/spaces/OOTS/pages/930450665/Two+systems+one+vision

[6] European Commission. *OOTS Common Services*. Retrieved from https://code.europa.eu/oots/common-services

[7] EU Training. *How to get an EPSO / CAST job*. Retrieved from https://eutraining.eu/content/how-get-epso-cast-job

[8] SprintCV. *DIGIT TM III framework contract: the new European Commission's IT services procurement*. Retrieved from https://www.sprintcv.com/blog/digit-tm-iii-framework-contract-the-new-european-commissions-it-services-procurement/

[9] A. C. Edmondson. *The Fearless Organization*. Wiley, 2018.

[10] A. C. Edmondson. Psychological safety and learning behavior in work teams. *Administrative Science Quarterly*, 44(2):350–383, 1999.

[11] N. Bennett and G. J. Lemoine. What a difference a word makes: understanding threats to performance in a VUCA world. *Business Horizons*, 57(3):311–317, 2014.

[12] J. Cascio. Facing the age of chaos. *Medium*, 2020.

[13] R. C. Mayer, J. H. Davis and F. D. Schoorman. An integrative model of organizational trust. *Academy of Management Review*, 20(3):709–734, 1995.

[14] N. Forsgren, J. Humble and G. Kim. *Accelerate*. IT Revolution Press, 2018.

[15] A. Bacchelli and C. Bird. Expectations, outcomes, and challenges of modern code review. In *Proc. ICSE*, pages 712–721, 2013.

[16] G. Avelino, L. Passos, A. Hora and M. T. Valente. A novel approach for estimating truck factors. In *Proc. ICPC*, pages 1–10, 2016.

[17] J. P. Kotter. *Leading Change*. Harvard Business Review Press, 1996.

[18] McKinsey & Company. Enduring Ideas: The 7-S Framework. *McKinsey Quarterly*, 2008.

AI Usage Notice

This report was prepared with the assistance of an AI writing assistant (Junie, by JetBrains, using the Claude Sonnet 5.5 model). The author provided the source material (text and information in the document), project context and reviewed the output. The author takes full responsibility for the content. The organization's information is public.